

Deployment Readiness & Go-Live Plan

MVP status, Community and Translation Team assessment — where the platform stands and what comes next

Project: Bunny Library — Arabic-first reading platform (مكتبة باني)

Baseline: Release E committed on main (0515e16)

Evidence: typecheck, lint and production build re-run 28 September 2026

Audience: Founder — go-live decision and continuation plan

Table of Contents

1. Executive Summary.....	1
2. Project Status Snapshot and Verification Evidence.....	2
2.1 Release history.....	2
2.2 Fresh verification results (28 September 2026).....	3
2.3 Seeded content inventory.....	3
2.4 Where the records live.....	4
3. MVP Completeness Assessment.....	4
3.1 What the MVP was defined to include.....	4
3.2 Beyond the MVP.....	5
3.3 Explicitly deferred — absent on purpose, not missing.....	5
3.4 Production hardening still outstanding.....	6
4. Community Features — Built or Not?.....	6
5. Translation Team — Built or Not?.....	7
5.1 Current state.....	7
5.2 Foundations already in place.....	7
5.3 The legal boundary.....	7
5.4 Proposed design — Release G, Translation Team lite.....	8
6. Deployment Readiness Audit.....	8
6.1 What is already green.....	9
6.2 Blockers and pending items.....	9
6.3 Go-live runbook (condensed).....	9
6.4 Post-deployment verification chain.....	10
7. Continuation Plan — Phases 0 to 3.....	11
7.1 Phase 0 — Go-live unblock.....	12
7.2 Phase 1 — Release F, production hardening (one engineering session).....	12

7.3 Phase 2 — Release G, Translation Team lite (one engineering session, gated).....	12
7.4 Phase 3 — Growth backlog (trigger-driven).....	13
8. Risks, Constraints and Accepted Limitations.....	13
9. Conclusions and Immediate Next Actions.....	14

Note: this table of contents is generated via field codes. After editing the document, right-click the table and choose "Update Field" to refresh page numbers.

1. Executive Summary

This report answers the four questions raised by the founder: is the project ready, are we ready to deploy, is the MVP finished or still missing features, and what is the status of the Community and the Translation Team. The short verdict: the platform is engineering-complete and deploy-ready. Every release from A through E is committed on the main branch with a clean working tree, and the full verification suite — typecheck, lint and a production build — was re-run on 28 September 2026 and passed with zero errors. Two items stand between the current state and a live deployment: the founder-side rotation of the Neon database credentials, and the creation of a git remote so the repository can be imported by the hosting provider.

The MVP question can be answered precisely because the prototype scope was contractually fixed in docs/PROTOTYPE_SCOPE.md before the first build: every feature in that inventory has shipped and has been browser-verified across the five releases. What remains is not MVP functionality but production hardening — rate limiting, password recovery, monitoring activation and legal document templates — plus a short list of features that were explicitly deferred by design, such as payments and downloads, both of which sit behind a lawyer gate.

Community features are built: series- and chapter-level comments, comment likes, five-star ratings with live aggregates, a report-and-moderation queue, and a complete notification center shipped in Releases B and E and were re-verified in the Release E end-to-end run. The Translation Team is the one genuine gap: no translation workflow exists in the codebase today beyond a translator byline field, and the scope document had already classified a translation pipeline as production-preparation rather than prototype scope. A concrete, costed proposal for a Release G translation workspace is included in this report.

Table 1: Founder questions and verdicts

Question	Verdict	Basis
Is the project ready?	Yes — engineering-complete	Releases A-E committed, clean tree, full verification re-run green today
Are we ready to deploy?	Almost — one founder blocker	Rotated Neon credentials plus a git remote; the documented runbook takes about 30 minutes
Is the MVP finished?	Yes, per the approved scope	All scope items shipped and browser-verified; the remainder is hardening and legal-gated work

Question	Verdict	Basis
Community — built?	Yes	Comments, likes, ratings, moderation and notifications (Releases B and E)
Translation Team — built?	No — proposal ready	Byline metadata only; a costed Release G design is included in this report

2. Project Status Snapshot and Verification Evidence

Bunny Library is an Arabic-first, right-to-left, dark-mode reading platform prototype for manga, webtoons and novels, built with Next.js 16 (App Router), TypeScript strict, Tailwind CSS v4, Prisma on Neon PostgreSQL, and NextAuth v4. All content is 100 percent original fiction, and the platform carries a binding demo disclosure in the footer and on series pages. This chapter records what exists today and the fresh evidence gathered specifically for this report, so that the deployment decision in Chapter 6 rests on re-verified facts rather than on earlier claims.

2.1 Release history

Five releases have been delivered since the project started, each closed with a report document, a browser-verified end-to-end QA run and a git commit. The table below summarizes the arc from the first reader experience to the current Release E baseline, which is the commit this report audited.

Table 2: Release history from reader core to community platform

Release	Focus	Commit
A	Reader core: home, explore, series pages, manga and webtoon readers, authentication, guarded admin panel, 6-series seed	early main
B	Novel reader, personal library, updates feed, full profile, community (comments, ratings, reports, moderation), admin collections, users and dashboard, chapter scheduler, 12-series seed	a64c1fa
C	Polish and production preparation: hydration fix, Arabic error boundaries, loading skeletons, security audit and headers, monitoring placeholders, deploy runbook	6148d90 (+ hardening 4cd1d0a)
D	Personalization: explainable For You rail, reading streaks, statistics and badges, embedded PostgreSQL 18 QA infrastructure	e49c473

Release	Focus	Commit
E	Notifications center, chapter comments with comment likes, Arabic smart search, RTL share cards, creator studio lite	0515e16

2.2 Fresh verification results (28 September 2026)

The following checks were executed on the repository as it stands today, specifically for this report. They confirm that the Release E state is healthy and reproducible, and they expose the two mechanical deployment prerequisites — the missing credentials and the missing git remote — that Chapter 6 assigns to owners.

Table 3: Verification run performed for this report

Check	Result
Typecheck (tsc --noEmit)	0 errors
ESLint	0 errors; 2 pre-existing warnings (legacy navigation pattern inside the manga reader)
Production build	Green — standalone output, every database-backed route dynamic, middleware active
Prerendered HTML documents	Zero — a strict CSP requirement; every document is rendered per request
Working tree	Clean; all work committed on main
Git remote	None configured — deployment prerequisite
Original art assets tracked in git	307 files under public/art

2.3 Seeded content inventory

The idempotent seed produces a lived-in demo with real community activity, and re-running it twice produces byte-identical counts, which is the property that makes the runbook safe to repeat. These are the numbers a freshly deployed instance will show immediately after the seed step.

Table 4: Seed output (verified idempotent across two consecutive runs)

Entity	Count	Entity	Count
Demo profiles (admin, editor, reader)	3	Comments	24
Series (manga, webtoon, novel)	12	Ratings	21
Chapters	46	Moderation reports	2

Entity	Count	Entity	Count
Chapter pages (original art)	287	Comment likes	8
Editorial collections	3	Notifications	4
Library items	7	Analytics events	348
Reading progress rows	6	Arabic prose chapters (novels)	12

2.4 Where the records live

Every claim in this report traces to a document in the repository, which keeps the audit trail continuous for any future collaborator. The most relevant records are listed below.

Table 5: Key project documents and their purpose

Document	Purpose
docs/PROTOTYPE_SCOPE.md	Binding MVP feature inventory, non-goals, roles, routes and acceptance criteria
docs/RELEASE_PLAN.md and RELEASE_B/C/D/E_REPORT.md	Per-release scope, QA evidence tables and honest limitations
docs/SECURITY_AUDIT.md	Hardening record, accepted risks and the credential incident note
docs/FOUNDER_DECISIONS.md	Decision log FD-1 through FD-13, including legal guardrails
README.md	Setup, deploy runbook, demo accounts, verification commands and known limitations

3. MVP Completeness Assessment

3.1 What the MVP was defined to include

The MVP definition was fixed in docs/PROTOTYPE_SCOPE.md before any build started: a public reader experience (items P-01 to P-13), a seven-screen admin panel (A-01 to A-07) and five platform items (S-01 to S-05), each governed by acceptance criteria covering RTL correctness, responsive behaviour at 360, 768 and 1280 pixels, accessibility, and legal safety. Because the scope also fixed the explicit non-goals, the question of missing features can be answered with evidence rather than opinion.

Table 6: Approved prototype scope and delivery status

Scope block	Items	Status
Public reader experience	P-01 to P-13 (home, explore, series, three readers, library, updates, profile, auth, mobile navigation, demo disclosure)	All shipped in Releases A-B and re-verified since
Admin panel	A-01 to A-07 (dashboard, series management, chapters and workflow, moderation, collections, users and roles)	All shipped in Releases A-B
Platform	S-01 to S-05 (schema, authorization, seed, original art, design system)	Shipped with one documented pivot (below)

One pivot is recorded honestly rather than hidden: the original plan named Supabase with database-level row-level security for the schema and authorization items, and the platform instead migrated to Neon PostgreSQL with versioned Prisma migrations under founder decision D-28. Row-level authorization is therefore enforced by the application in three layers — middleware, the admin layout guard, and requireRole checks inside every admin query and server action — a trade-off documented in the README enforcement table. Every other platform item, including the idempotent 12-series seed, the generated original art and the RTL design system, shipped as specified.

3.2 Beyond the MVP

Two further releases delivered features that were never part of the MVP inventory and were approved later through the value-proposition menu. Release D added zero-schema personalization: a For You rail whose every card carries an Arabic reason anchor derived from the reader's own history, plus reading streaks, statistics and eight deterministic badges. Release E shipped the entire remaining menu in a single migration: the notification center, chapter-level comments with comment likes, Arabic smart search with normalization and typo tolerance, RTL share cards with per-series Open Graph images, and a read-only creator studio for editors. Taken together, the platform today is well past its original MVP definition.

3.3 Explicitly deferred — absent on purpose, not missing

Several capabilities are absent by design, and listing them prevents them from being mistaken for gaps. Each deferral carries a rationale that was accepted when the scope was fixed and restated in every release report since.

Table 7: Deferred-by-design items and their rationale

Feature	Status	Rationale
Payments and subscriptions	Deferred behind a lawyer gate	No purchase flow may exist without legal review; nothing purchasable is reachable anywhere in the product
Licensed or real content	Forbidden by guardrail	100 percent original fiction is binding; scope extends this to public-domain translations
Downloads and offline files	Deferred	Legal exposure plus out of prototype scope
Social graph (follows, direct messages, forums)	Deferred	Moderation surface grows non-linearly; comments are the approved community scope
Multi-language interface	Deferred (D-02)	Arabic-first is the product identity
Mobile applications	Deferred	Web prototype first
Email verification and OTP	Prototype non-goal	Documented demo accounts only

3.4 Production hardening still outstanding

Finally, there is a short list of items a real user base would want on day one but a prototype does not require, and none of them block a deployment of the current demo. They are scheduled in the continuation plan in Chapter 7: login and comment rate limiting, password recovery (which needs an email-channel decision), monitoring activation behind the existing Sentry and PostHog stubs, a consent banner, and final PDPL, Terms of Service and DMCA template language, which requires a lawyer. The verdict of this chapter is therefore firm: the MVP is complete relative to the approved scope, and the honest label for everything remaining is hardening and legal preparation, not missing MVP features.

4. Community Features — Built or Not?

Yes — the community layer is built, shipped and browser-verified. It was delivered in two passes: Release B introduced comments, ratings, reporting and moderation, and Release E completed the loop with comment likes, chapter-scoped commenting inside all three readers, and a full notification center. Every feature in the table below was re-verified end to end in the Release E QA run against a real PostgreSQL 18 instance, including optimistic interface behaviour and database-backed state.

Table 8: Community feature matrix

Feature	Where it lives	Shipped in
Comments, series scope	Series page discussion thread below the chapter list	B
Comments, chapter scope	All three readers: inline at the chapter end for webtoon; header-trigger dialog for manga and novel	E
Comment likes	Every comment; one like per profile per comment enforced by a compound unique index; optimistic toggle with rollback	E
Ratings	Series pages; five stars with aggregates recomputed on every write	B
Report and moderation	Report flow on any comment; admin queue with flagged, hidden and visible states; hide, restore and delete (delete is admin-only)	B
Notification center	Header bell with unread badge and dropdown feed, full notifications page, mark-read on click and mark-all; server-side fan-out on chapter publish and on comment likes	E
Sharing	Share sheet on every series (native share, copy link, X, WhatsApp, Telegram) plus a branded per-series Open Graph image	E

The seed makes this community look real from the first minute: 24 comments including flagged and hidden examples for the moderation demo, 21 ratings, 8 comment likes, 4 notifications for the demo reader in a mix of read and unread states, and 2 moderation reports. Signing in as the demo reader immediately shows an unread bell badge, an active notification feed, and comment threads that can be liked and replied to.

It is equally important to state what community does not include, because it was excluded on purpose rather than missed. There is no social graph — no follows, direct messages or forums — since the scope document flagged the non-linear moderation cost, and comments remain the approved community boundary. The notification bell polls every 60 seconds rather than using push, publish fan-out is capped at 500 profiles per trigger with no retry queue, and search relevance is computed in memory; all three are prototype-appropriate choices with documented upgrade paths in Chapter 7.

5. Translation Team — Built or Not?

5.1 Current state

No. The Translation Team does not exist in the codebase, and this is the one direct answer in this report that is negative. The only trace of the concept is a translator byline: the series table carries an optional translator field, the admin series forms can edit it, and public series pages render it as a credit line under the author. That is display metadata only — there is no workflow, no role, no queue and no data model for translated content anywhere in the platform. This is consistent with the plan rather than an oversight: docs/PROTOTYPE_SCOPE.md explicitly lists a content translation workflow, described there as drafting chapters in a source language followed by a translation queue, under its Production-Preparation heading, meaning architecturally anticipated but deliberately not built. The platform has three roles today — reader, editor and admin — and none of them represents a translator.

5.2 Foundations already in place

Although nothing was built, the platform grew into a strong starting position, because most of the plumbing a translation pipeline needs already exists and is production-tested. The table below maps each existing foundation to its use in the proposed design.

Table 9: Existing foundations and how a translation team would use them

Foundation	What exists today	Use in the translation pipeline
Chapter workflow	Native draft, review and published states on every chapter	Maps directly onto translated, reviewed and published states
Roles and authorization	Editor role with requireRole enforced in every admin query and server action	Permission plumbing for translator-scoped surfaces
Publication scheduler	Scheduled timestamps with a public coming-soon badge	Release cadence for translated chapters
Creator studio	Read-only per-series pipeline and top-chapter statistics	Extend with translation progress widgets
Server actions with validation	Twelve schema-validated write actions in production	The established safe-write pattern for new actions
Notifications	Batched fan-out with a typed enum	Assignment and review notifications reuse it directly

5.3 The legal boundary

Any translation plan must respect the standing guardrails, which are binding: no real or licensed content of any kind — a rule the scope document extends explicitly to public-domain translations — and no change to that posture without lawyer review. In practice, a

Translation Team today can work only on the platform's own original Arabic fiction, for example producing English editions of the four seeded novels, or prepare the pipeline so it is ready the day licensing is legally cleared. Translating third-party works, even public-domain ones, requires the lawyer gate first. This boundary directly shapes the proposal below and should be part of the founder's direction decision.

5.4 Proposed design — Release G, Translation Team lite

The proposal keeps the release discipline of Releases D and E: one migration at most, seed additions under the same idempotency guards, a browser-verified end-to-end run, and a release report with README updates. The first decision for the founder is the role strategy, and the recommendation is option B for the first pass.

Table 10: Role strategy options for the translation team

Option	Approach	Migration cost	Trade-off
A	Add a translator value to the user role enum and build translator-scoped surfaces	One migration	Cleanest long-term permissioning, but touches the role enum and every role selector
B	Recommended: reuse the editor role and scope translation actions to assigned translators in the application layer	Zero schema change	Ships fastest; permissioning is enforced in code exactly as admin authorization is today

The lite scope would then add one translation table holding the chapter reference, target language, body text, workflow status, translator, reviewer and timestamps; an assignment queue in the admin area; a side-by-side source and translation editor for novel prose; a small glossary term base; translation progress widgets in the creator studio; and assignment notifications reusing the existing fan-out. Workflow states follow the existing draft-review-published semantics, so moderation, scheduling and publishing need no new concepts. The estimate is one working session at the established quality bar, comparable in size to Release E.

6. Deployment Readiness Audit

This chapter converts the status evidence into a deployment decision. The conclusion up front: the codebase is deploy-ready, the runbook is written and documented in the README, and the remaining work is a short, owner-assigned checklist rather than engineering uncertainty. Nothing in the audit surfaced a defect; the findings are logistical.

6.1 What is already green

Table 11: Deployment readiness — verified green areas

Area	Evidence
Code health	Typecheck 0 errors; lint 0 errors with 2 pre-existing warnings; production build green with standalone output
Security	Nonce-based strict-dynamic CSP verified 13 of 13 checks; security headers on every route; dependency audit removed 46 unused packages; production vulnerabilities at 6 high, 0 critical
Schema and data	Four versioned migrations applied and verified on real PostgreSQL 18; failsafe checks prove the application fails loudly on wrong or missing database URLs
Content pipeline	Seed verified idempotent twice over; 307 original art assets committed, so a fresh clone renders completely without regeneration
Operations	Health endpoint with liveness and database probes; rollback paths documented for both migrations and deployments
Documentation	Deploy runbook targeting a 30-minute clone-to-live path; five release reports; security audit; founder decision log

6.2 Blockers and pending items

Six items remain, and only the first two genuinely gate the deployment. Each has an owner and an effort estimate; none requires new engineering decisions.

Table 12: Open items with owners and effort

#	Item	Owner	Effort	Notes
1	Rotate the Neon credentials and deliver both connection strings — pooled to DATABASE_URL, direct to DIRECT_URL — through the secure channel	Founder	About 15 minutes	The database itself is unaffected; rotation closes the incident recorded in the security audit
2	Create the git remote (GitHub or GitLab) and push main, or grant an existing target	Founder	About 10 minutes	The hosting import requires a remote; none is configured today
3	Extend the database constraint verifier for the Release E tables and enum	Engineering	About 30 minutes	The verifier covers 41 checks from earlier releases; notifications and comment likes need coverage before the post-migration run

#	Item	Owner	Effort	Notes
4	Complete the deferred strict-CSP end-to-end pass (sign-in POST and server actions against a live database)	Engineering	About 15 minutes after item 1	Blocked only by credentials; everything else was verified
5	Decide on monitoring activation (Sentry, PostHog) and provide keys, or keep placeholders	Founder	Decision only	Stubs are wired with one-step activation notes
6	Set the production origin (NEXTAUTH_URL and the domain)	Founder	With hosting setup	Required by authentication in production

6.3 Go-live runbook (condensed)

The full runbook lives in the README and targets a new-developer clone-to-live path of thirty minutes. In condensed form, with owners from the table above, the sequence is:

1. Neon: create or open the project and copy both connection strings — pooled (pooler toggle on) and direct (toggle off).
2. Secrets: write them into the local environment file (never committed); add DATABASE_URL, DIRECT_URL, NEXTAUTH_SECRET and NEXTAUTH_URL in the hosting project settings.
3. Schema: install dependencies, generate the Prisma client, apply the versioned migrations using the direct URL.
4. Content: run the seed twice — the second run must change nothing; art is already committed, regeneration is optional.
5. Deploy: import the repository at the hosting provider, confirm the four environment variables, and deploy the standalone build.
6. Smoke: open the home page (12 series, right-to-left), check the health endpoint, sign in as each demo account, and confirm the admin dashboard.

6.4 Post-deployment verification chain

After the rotated credentials arrive, the same chain restores full database QA locally before deployment, and it repeats verbatim against production afterwards. Expected outcomes are listed so any deviation is immediately visible:

1. Migration status: four migrations recognized; the deploy run applies them, and a second run reports the database as up to date.
2. Constraint verifier, after its Release E extension: every check passes.

3. Seed twice: byte-identical counts, including 348 analytics events, 24 comments, 8 comment likes and 4 notifications.
4. Spot checks: the For You rail with Arabic reason anchors, profile streak and badges, the notification bell, an Arabic search typo case, a per-series Open Graph image, and one moderation action.
5. Strict-CSP completion: sign-in POST and a server action exercised in a real browser with zero console violations.

7. Continuation Plan — Phases 0 to 3

The plan below is sequenced so that deployment is unblocked first, hardening lands immediately after, and the Translation Team ships as the next feature release. The founder's approval of this sequence, plus the two direction decisions in it, should be recorded as FD-14 in docs/FOUNDER_DECISIONS.md, keeping the decision log continuous from FD-1.

7.1 Phase 0 — Go-live unblock

Phase 0 exists to convert the blocker list into a live URL. It needs roughly one hour of combined effort once the rotated credentials exist, and it ends with the prototype publicly reachable, fully seeded and verified in its production environment.

Table 13: Phase 0 steps, owners and outputs

Step	Owner	Output
Deliver rotated pooled and direct connection strings	Founder	Credentials in the local environment file, never in chat
Extend the constraint verifier for Release E tables	Engineering	Verifier ready for the post-migration run
Create the remote and push main	Founder and engineering	Repository importable by the hosting provider
Run the post-deployment verification chain locally	Engineering	Migrations applied; seed identical twice
Deploy and run the smoke checks	Founder and engineering	Production URL with a healthy endpoint
Complete the strict-CSP end-to-end pass in production	Engineering	Zero console violations on sign-in and server actions

7.2 Phase 1 — Release F, production hardening (one engineering session)

Release F hardens the platform for real users without changing scope. The work items, in priority order, are: login and comment-posting rate limiting at the middleware and action layers; per-type notification preferences on the profile; deterministic tie-breaking in the For You rail by ordering anchors on recency, which is a one-line change; timezone-aware streak buckets once real users exist; cleanup of the two manga-reader lint warnings; an admin password-change capability; and monitoring activation behind the existing stubs if keys are provided. Exit criteria mirror the house convention: typecheck and lint clean, production build green, browser-verified end to end, report and README updated, one commit.

7.3 Phase 2 — Release G, Translation Team lite (one engineering session, gated)

As designed in Chapter 5, this phase is gated on two founder decisions: the direction — English editions of original fiction first, or pipeline preparation only — and the role strategy, where option B reuses the editor role with zero schema change. Deliverables follow the D and E pattern: the translation table and assignment queue, the side-by-side editor for novel prose, studio progress widgets, assignment notifications, seed additions under the idempotency guards, a browser-verified end-to-end run, and a release report.

7.4 Phase 3 — Growth backlog (trigger-driven)

The backlog items below stay parked until their trigger fires, which keeps the roadmap honest about what the current scale does not yet need.

Table 14: Growth backlog and start triggers

Item	Trigger to start
Database-backed search index (trigram or full-text)	Catalog grows beyond the low thousands of series
Push notifications replacing 60-second polling	Retention data shows notification-driven return visits
Image CDN and responsive variants	Real traffic on the art assets
PDPL, Terms of Service, DMCA templates and consent banner	Public launch at scale, with the lawyer
Social graph (follows and feeds)	Founder scope decision — moderation surface grows non-linearly
Mobile applications	Remains deferred; web first

8. Risks, Constraints and Accepted Limitations

Every limitation below is already documented in the repository rather than discovered here, which is the correct posture for a prototype heading to deployment. Severity labels reflect the demo context; a production launch with real users would raise several of them, and the continuation plan already schedules the corresponding work.

Table 15: Risk register with mitigation status

Risk	Severity	Mitigation and status
No database-level row-level security; authorization enforced in the application (middleware, layout guard, requireRole)	Accepted	Documented enforcement table in the README; keep credential hygiene and activate monitoring in Phase 1
CSP style source retains unsafe-inline (framework critical CSS and component style attributes)	Low	Scripts are strictly nonce-gated and verified 13 of 13; residual exposure is style injection only
Open Graph image renderer has no bidi support; Arabic strings are word-reversed as a workaround	Low	Pure-Arabic rows visually verified; mixed-direction runs use direction-agnostic formatting; a headless-browser renderer is the documented upgrade if ever needed
Notification fan-out capped at 500 profiles per publish with no retry queue	Low at demo scale	Queue infrastructure sits in the growth backlog
Bell uses 60-second polling, not push	Low	Documented upgrade path; acceptable while the audience is small
Search relevance is computed in memory	Low at demo scale	Normalization is isolated and portable to SQL; index in the backlog
Neon credentials pending rotation since the sandbox-reset incident	Blocker — founder	Database unaffected; the verification chain is documented and rehearsed on embedded PostgreSQL 18
Strict-CSP sign-in and server-action end-to-end deferred pending credentials	Low	Scheduled as the final Phase 0 step
Single-founder dependency for credentials, domain and monitoring decisions	Process risk	All remaining founder actions take minutes and carry owners in Chapter 6

9. Conclusions and Immediate Next Actions

The platform is in the strongest state of its history: five releases committed, the full verification suite green on the day of this report, a documented deploy runbook, and honest

written records of every accepted limitation. The four founder questions therefore receive direct answers, and none of the remaining work is engineering uncertainty — it is a short list of founder-side actions followed by two well-scoped sessions.

Table 16: Final verdicts on the founder questions

Question	Answer
Is the project ready?	Yes — Releases A through E complete, clean tree, verification re-run green on 28 September 2026
Are we ready to deploy?	Yes, pending two items: rotated Neon credentials and a git remote — about one hour of combined effort
Is the MVP finished?	Yes, per the approved prototype scope; the remainder is hardening and legal-gated work, not MVP features
Was the Community made?	Yes — comments, chapter comments, likes, ratings, moderation and notifications, all browser-verified
Was the Translation Team made?	No — byline metadata only; a costed Release G proposal is ready and needs two decisions

The immediate next actions, in order, with owners:

1. Founder: rotate the database owner password and deliver the pooled and direct connection strings through the secure channel — never in chat.
2. Founder: create the git remote, or approve an existing target, and authorize the push of main.
3. Founder: approve the Phase 0-3 sequence and choose the Translation Team direction and role option, recorded as FD-14.
4. Engineering: on receipt of the first two items, execute Phase 0 end to end and start Release F immediately after.

From there the platform moves from a verified prototype to a deployed one without new scope, and the next feature release — the Translation Team — starts from foundations that already exist. The decision log, release reports and runbook in the repository carry everything a future collaborator needs to continue exactly where this report ends.